# Professional AC Core Cutting Business Website

Database Schema Document • v1.0 • Production Baseline

| ITEM | DATABASE DECISION |
|---|---|
| System | AC Core Cutting Business Website |
| Database model | Relational database |
| Recommended engine | PostgreSQL |
| Application | Next.js + TypeScript server-rendered / hybrid website |
| Primary purpose | Structured business content + secure enquiry/lead persistence |
| Core entities | Business, Services, Service Areas, Gallery, Reviews, FAQs, Enquiries, Analytics |
| MVP boundary | No accounts, payments, advanced scheduling or full CMS |

**Design principle: the database should be normalized, secure, easy to migrate and simple enough for the MVP, while preserving clear extension points for future lead management, scheduling, WhatsApp workflows and CMS features.**

Where the project documents define a requirement but not an exact database implementation, this document labels the database-engineering choice as a recommendation.

# 1. Database Scope

The project is a local-service lead-generation website whose primary conversion paths are phone, WhatsApp and quote requests. The approved technical architecture uses server-side/API-backed form handling and a structured content model for business information, services, gallery, reviews, FAQs and service areas.

The technical requirements define the content model as Business, Service, GalleryItem, Review, FAQ and ServiceArea, and define a POST enquiry flow containing name, phone, WhatsApp preference, service type, location and message.

## Database responsibilities

• Store verified public business information.

• Store and publish the approved service catalogue.

• Store genuinely served service areas.

• Store metadata for real project gallery images.

• Store approved testimonials/reviews.

• Store customer-facing FAQs.

• Persist quote/enquiry leads securely.

• Support privacy-conscious conversion analytics.

• Provide versioned migrations and reproducible environments.

**The database is not the presentation layer. Public pages should consume controlled server-side queries rather than exposing a general-purpose database API to the browser.**

# 2. Recommended Database Technology

| AREA | RECOMMENDATION | REASON |
|---|---|---|
| Engine | PostgreSQL | Strong relational integrity, JSONB support and production maturity |
| IDs | UUID | Stable identifiers without exposing sequential record counts |
| Timestamps | TIMESTAMPTZ | Correct handling of deployment/user time zones |
| JSON fields | JSONB | Useful for ordered benefits/process steps and limited flexible metadata |
| Migrations | Version-controlled SQL/ORM migrations | Reproducible local → staging → production changes |
| Queries | Parameterized SQL / safe ORM | Prevents injection and centralizes data access |
| Connection | Server-only DB credentials | Database must never be directly reachable from client code |

Important: PostgreSQL is a proposed implementation choice. The project TRD recommends Next.js + TypeScript and a server/API-backed form architecture but does not mandate a specific database engine or ORM.

## Environment separation

| ENVIRONMENT | DATABASE | RULE |
|---|---|---|
| Local | Local/dev PostgreSQL | Safe seed/demo data only |
| Preview/Staging | Separate staging database | Never use production customer leads |
| Production | Protected production database | Restricted credentials, backups and monitoring |

The technical requirements define Local, Preview/Staging and Production as separate purposes; staging is intended for content, UI, forms and QA before production.

# 3. Complete Entity Inventory

| TABLE | TYPE | PURPOSE | PRIORITY |
|---|---|---|---|
| business_profile | Core | Single source of business identity/contact data | MVP |
| services | Core | Service catalogue and SEO metadata | MVP |
| service_areas | Core | Genuine operating/service locations | MVP |
| gallery_items | Core | Real project photos and metadata | MVP |
| reviews | Core | Approved customer testimonials/reviews | MVP |
| faqs | Core | Customer questions and answers | MVP |
| enquiries | Transactional | Quote/contact lead records | MVP |
| analytics_events | Operational | Approved conversion events | MVP |
| site_settings | Configuration | Small non-secret application settings | Optional |

## Normalization target

The model follows a practical normalized relational design: repeated business/service/location data is stored once and referenced by foreign keys. Flexible arrays or metadata are used only where they simplify content structures that do not need independent relational querying.

## Core relationship map

```
business_profile 1 ■■ N services
business_profile 1 ■■ N service_areas
services 1 ■■ N gallery_items
services 1 ■■ N faqs
services 1 ■■ N enquiries
service_areas 1 ■■ N enquiries
business_profile 1 ■■ N reviews
business_profile 1 ■■ N analytics_events
```

# 4. business_profile

This table stores the business identity and public contact data that should remain consistent across the website.

| COLUMN | TYPE | NULL | KEY / RULE |
|---|---|---|---|
| id | uuid | NO | PK; default gen_random_uuid() |
| business_name | varchar(160) | NO | Verified public name |
| logo_url | text | YES | Public asset path/URL |
| phone | varchar(32) | NO | Primary contact number |
| whatsapp | varchar(32) | YES | WhatsApp number if available |
| address | text | YES | Public address if applicable |
| city | varchar(100) | NO | Main operating city |
| hours | jsonb | YES | Structured opening hours |
| description | text | YES | Approved business description |
| google_business_url | text | YES | Google Business Profile/review URL |
| created_at | timestamptz | NO | Record creation |
| updated_at | timestamptz | NO | Last modification |

## Integrity

• Maintain one active business profile for the single-business MVP.

• Do not fabricate phone, address, hours, service-area or experience claims.

• Use owner-approved values for all public business facts.

• Keep Google Business Profile URL optional because it may not be available at launch.

**The project requirements explicitly state that business-specific phone, WhatsApp, address, service areas, hours and claims must come from the owner.**

# 5. services

The approved service structure contains six categories: AC Core Cutting, RCC Core Cutting, AC Drain Hole, Concrete Wall Drilling, Pipe & Cable Passage and Other Services.

| COLUMN | TYPE | NULL | KEY / RULE |
|---|---|---|---|
| id | uuid | NO | PK |
| slug | varchar(120) | NO | UNIQUE; stable public identifier |
| name | varchar(160) | NO | Display name |
| summary | varchar(300) | NO | Service-card summary |
| description | text | NO | Detailed customer-facing content |
| benefits | jsonb | YES | Ordered benefit list |
| process | jsonb | YES | Ordered process steps |
| image_url | text | YES | Primary service image |
| seo_title | varchar(180) | YES | Unique SEO title |
| seo_description | varchar(320) | YES | Unique SEO description |
| is_published | boolean | NO | Public visibility |
| display_order | integer | NO | Sort order |
| created_at | timestamptz | NO | Creation |
| updated_at | timestamptz | NO | Modification |

## Indexes / constraints

UNIQUE(slug); INDEX(is_published, display_order); CHECK(display_order >= 0).

Public service slugs should remain stable after launch because they form part of indexable URLs.

# 6. service_areas

Service areas define where the business genuinely operates. The database must not be used to create misleading location coverage for SEO.

| COLUMN | TYPE | NULL | KEY / RULE |
|---|---|---|---|
| id | uuid | NO | PK |
| slug | varchar(120) | NO | UNIQUE |
| name | varchar(120) | NO | Area name |
| description | text | YES | Optional local context |
| map_reference | text | YES | Optional map link/reference |
| is_published | boolean | NO | Public visibility |
| display_order | integer | NO | Sort order |
| created_at | timestamptz | NO | Creation |
| updated_at | timestamptz | NO | Modification |

## Lead-location distinction

enquiries.location is customer-entered text and remains independent from service_areas. If the customer types an arbitrary location, the application must not silently create a new public service area.

## Indexes

UNIQUE(slug); INDEX(is_published, display_order).

# 7. gallery_items

Gallery records support real work images, including completed jobs, core cutting, AC installation holes and before/after examples.

| COLUMN | TYPE | NULL | KEY / RULE |
|---|---|---|---|
| id | uuid | NO | PK |
| image_url | text | NO | Optimized media path/URL |
| alt_text | varchar(300) | NO | Accessible description |
| title | varchar(180) | YES | Optional display title |
| category | varchar(80) | NO | Gallery category |
| location | varchar(120) | YES | Job location if appropriate |
| service_id | uuid | YES | FK → services.id |
| project_date | date | YES | Optional |
| is_published | boolean | NO | Public visibility |
| display_order | integer | NO | Sort order |
| created_at | timestamptz | NO | Creation |
| updated_at | timestamptz | NO | Modification |

Recommended FK: service_id REFERENCES services(id) ON DELETE SET NULL. Retiring a service should not destroy historical gallery content.

## Media integrity

• Real project photographs only for work-proof sections.

• Optimize image dimensions and modern formats before serving.

• Required alt_text for informative images.

• Do not expose private customer information in filenames or metadata.

# 8. reviews

Reviews are social-proof data. The database distinguishes the customer-facing content from its source and approval state.

| COLUMN | TYPE | NULL | KEY / RULE |
|---|---|---|---|
| id | uuid | NO | PK |
| customer_name | varchar(120) | NO | Approved display name |
| review_text | text | NO | Actual testimonial/review |
| rating | smallint | YES | CHECK 1–5 when present |
| source | varchar(60) | NO | e.g. manual / Google |
| source_url | text | YES | Public source link |
| review_date | date | YES | Date if known |
| approved | boolean | NO | Only approved reviews are public |
| created_at | timestamptz | NO | Creation |
| updated_at | timestamptz | NO | Modification |

Review policy: use real customer reviews/testimonials with appropriate attribution/permission. The database must never be seeded with fabricated testimonials.

## Public query

SELECT ... FROM reviews WHERE approved = true ORDER BY review_date DESC NULLS LAST, created_at DESC;

# 9. faqs

FAQs are customer-facing content and can be global or service-specific.

| COLUMN | TYPE | NULL | KEY / RULE |
|---|---|---|---|
| id | uuid | NO | PK |
| question | varchar(300) | NO | Customer question |
| answer | text | NO | Approved answer |
| category | varchar(80) | YES | Optional category |
| service_id | uuid | YES | FK → services.id |
| display_order | integer | NO | Sort order |
| is_published | boolean | NO | Public visibility |
| created_at | timestamptz | NO | Creation |
| updated_at | timestamptz | NO | Modification |

## Examples supported by the project

• What is core cutting?

• What size hole is required?

• Can you drill RCC walls?

• How long does it take?

• Which areas do you serve?

Service-specific FAQs use service_id; global FAQs keep service_id NULL.

# 10. enquiries — Lead Database

This is the primary transactional table. It persists customer requests after server-side validation and spam controls.

| COLUMN | TYPE | NULL | KEY / RULE |
|---|---|---|---|
| id | uuid | NO | PK |
| name | varchar(120) | NO | Required |
| phone | varchar(32) | NO | Required; validated |
| whatsapp_preference | boolean | YES | Preferred contact channel |
| service_id | uuid | YES | FK → services.id |
| location | varchar(160) | NO | Customer-entered service location |
| message | text | YES | Requirement/context |
| status | varchar(30) | NO | new/contacted/quoted/closed/spam |
| source_page | varchar(255) | YES | Origin page |
| created_at | timestamptz | NO | Lead creation |
| updated_at | timestamptz | NO | Last state change |

## Recommended status lifecycle

new → contacted → quoted → closed
new → spam

## Indexes

INDEX(created_at DESC); INDEX(status, created_at DESC); INDEX(service_id, created_at DESC).

**These indexes support future lead dashboards without changing the core lead model.**

# 11. analytics_events

The database can persist a minimal event stream for approved conversion tracking, although a dedicated privacy-conscious analytics provider may be preferable for high-volume page analytics.

| COLUMN | TYPE | NULL | KEY / RULE |
|---|---|---|---|
| id | uuid | NO | PK |
| event_name | varchar(60) | NO | Allowlisted event |
| page_path | varchar(255) | YES | Current page |
| service_id | uuid | YES | Optional context |
| metadata | jsonb | YES | Non-sensitive metadata only |
| occurred_at | timestamptz | NO | Event timestamp |

## Approved event names

page_view, phone_click, whatsapp_click, quote_start, quote_submit, map_click, service_cta_click

## Do not store

• Phone numbers in metadata.

• Full enquiry messages.

• Passwords or authentication data.

• Government IDs, payment data or unrelated personal information.

# 12. Optional site_settings

A small configuration table can be added for non-secret site-level settings.

| COLUMN | TYPE | RULE |
|---|---|---|
| key | varchar(80) | PK/UNIQUE |
| value | jsonb | Typed configuration |
| updated_at | timestamptz | Modification timestamp |

Examples: analytics_enabled, contact_form_enabled, maintenance_message, default_social_image, schema_version.

**Never store database passwords, API keys, SMTP secrets or other confidential credentials here. Secrets belong in environment variables or a secret manager and must never be sent to the browser.**

# 13. SQL DDL — PostgreSQL Baseline

```
CREATE EXTENSION IF NOT EXISTS pgcrypto; CREATE TABLE business_profile ( id uuid PRIMARY KEY DEFAULT gen_random_uuid(), business_name varchar(160) NOT NULL, logo_url text, phone varchar(32) NOT NULL, whatsapp varchar(32), address text, city varchar(100) NOT NULL, hours jsonb, description text, google_business_url text, created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL DEFAULT now() ); CREATE TABLE services ( id uuid PRIMARY KEY DEFAULT gen_random_uuid(), slug varchar(120) NOT NULL UNIQUE, name varchar(160) NOT NULL, summary varchar(300) NOT NULL, description text NOT NULL, benefits jsonb, process jsonb, image_url text, seo_title varchar(180), seo_description varchar(320), is_published boolean NOT NULL DEFAULT true, display_order integer NOT NULL DEFAULT 0 CHECK (display_order >= 0), created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL DEFAULT now() ); CREATE TABLE service_areas ( id uuid PRIMARY KEY DEFAULT gen_random_uuid(), slug varchar(120) NOT NULL UNIQUE, name varchar(120) NOT NULL, description text, map_reference text, is_published boolean NOT NULL DEFAULT true, display_order integer NOT NULL DEFAULT 0 CHECK (display_order >= 0), created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL DEFAULT now() );
```

# 14. SQL DDL — Content & Lead Tables

```
CREATE TABLE gallery_items ( id uuid PRIMARY KEY DEFAULT gen_random_uuid(), image_url text NOT NULL, alt_text varchar(300) NOT NULL, title varchar(180), category varchar(80) NOT NULL, location varchar(120), service_id uuid REFERENCES services(id) ON DELETE SET NULL, project_date date, is_published boolean NOT NULL DEFAULT true, display_order integer NOT NULL DEFAULT 0, created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL DEFAULT now() ); CREATE TABLE reviews ( id uuid PRIMARY KEY DEFAULT gen_random_uuid(), customer_name varchar(120) NOT NULL, review_text text NOT NULL, rating smallint CHECK (rating BETWEEN 1 AND 5), source varchar(60) NOT NULL, source_url text, review_date date, approved boolean NOT NULL DEFAULT false, created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL DEFAULT now() ); CREATE TABLE faqs ( id uuid PRIMARY KEY DEFAULT gen_random_uuid(), question varchar(300) NOT NULL, answer text NOT NULL, category varchar(80), service_id uuid REFERENCES services(id) ON DELETE SET NULL, display_order integer NOT NULL DEFAULT 0, is_published boolean NOT NULL DEFAULT true, created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL DEFAULT now() ); CREATE TABLE enquiries ( id uuid PRIMARY KEY DEFAULT gen_random_uuid(), name varchar(120) NOT NULL, phone varchar(32) NOT NULL, whatsapp_preference boolean, service_id uuid REFERENCES services(id) ON DELETE SET NULL, location varchar(160) NOT NULL, message text, status varchar(30) NOT NULL DEFAULT 'new' CHECK (status IN ('new','contacted','quoted','closed','spam')), source_page varchar(255), created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL DEFAULT now() );
```

# 15. SQL DDL — Analytics & Indexes

```
CREATE TABLE analytics_events ( id uuid PRIMARY KEY DEFAULT gen_random_uuid(), event_name varchar(60) NOT NULL, page_path varchar(255), service_id uuid REFERENCES services(id) ON DELETE SET NULL, metadata jsonb, occurred_at timestamptz NOT NULL DEFAULT now() ); CREATE INDEX idx_services_published_order ON services (is_published, display_order); CREATE INDEX idx_service_areas_published_order ON service_areas (is_published, display_order); CREATE INDEX idx_gallery_published_order ON gallery_items (is_published, display_order); CREATE INDEX idx_gallery_service ON gallery_items (service_id); CREATE INDEX idx_faqs_published_order ON faqs (is_published, display_order); CREATE INDEX idx_faqs_service ON faqs (service_id); CREATE INDEX idx_enquiries_created_at ON enquiries (created_at DESC); CREATE INDEX idx_enquiries_status_created_at ON enquiries (status, created_at DESC); CREATE INDEX idx_enquiries_service_created_at ON enquiries (service_id, created_at DESC); CREATE INDEX idx_analytics_event_time ON analytics_events (event_name, occurred_at DESC);
```

The DDL is a baseline implementation example. Exact migration syntax may vary depending on whether Prisma, Drizzle, Kysely, raw SQL or another data layer is selected.

# 16. Query Patterns

## Public services

```
SELECT id, slug, name, summary, description, benefits, process, image_url, seo_title, seo_description FROM services WHERE is_published = true ORDER BY display_order;
```

## Service by slug

```
SELECT * FROM services WHERE slug = $1 AND is_published = true LIMIT 1;
```

## Gallery

```
SELECT * FROM gallery_items WHERE is_published = true ORDER BY display_order, created_at DESC;
```

## Approved reviews

```
SELECT * FROM reviews WHERE approved = true ORDER BY review_date DESC NULLS LAST, created_at DESC;
```

## Latest leads

```
SELECT * FROM enquiries WHERE status <> 'spam' ORDER BY created_at DESC LIMIT $1;
```

## Service-specific FAQ

```
SELECT * FROM faqs WHERE is_published = true AND (service_id = $1 OR service_id IS NULL) ORDER BY display_order;
```

**All database queries must be parameterized or safely generated by the chosen ORM/query builder.**

# 17. Enquiry Write Transaction

Recommended backend sequence for a quote request:

***FORM → SERVER → SCHEMA VALIDATION → NORMALIZATION → RATE LIMIT → BOT CHECK → FK VALIDATION → INSERT ENQUIRY → LEAD DELIVERY → SAFE RESPONSE***

| STEP | DATABASE / SERVER ACTION | FAILURE BEHAVIOR |
|---|---|---|
| 1 | Receive POST over HTTPS | Reject non-supported request |
| 2 | Validate schema | 400 with field-level safe errors |
| 3 | Normalize phone/input | Reject malformed values |
| 4 | Apply rate limit / spam controls | 429 or silent safe rejection |
| 5 | Validate service_id if supplied | 400 if invalid |
| 6 | Insert enquiry | 500 safe generic error; log operational detail |
| 7 | Deliver to email/CRM/workflow | Retry/queue strategy if delivery is added |
| 8 | Return success | Generic confirmation only |

The project requirements specifically call for POST over HTTPS, required-field/phone/length validation, rate limiting or honeypot/CAPTCHA-equivalent protection, safe generic responses and operational logging without unnecessary sensitive information.

# 18. Data Lifecycle & Retention

| DATA | CREATE | UPDATE | DELETE / RETIRE |
|---|---|---|---|
| Business | Owner/admin | As needed | Update; normally do not delete |
| Services | Owner/admin | Content updates | Unpublish/retire rather than hard-delete public history |
| Service Areas | Owner/admin | Coverage changes | Unpublish when no longer served |
| Gallery | Owner/admin | Metadata changes | Unpublish/remove asset when appropriate |
| Reviews | Owner/admin | Approval/moderation | Unpublish if withdrawn or no longer authorized |
| FAQs | Owner/admin | Content updates | Unpublish |
| Enquiries | Customer submission | Business status | Retention policy; mark closed/spam |
| Analytics | Event submission | Append-only | Retention/aggregation policy |

**The project requirements emphasize minimizing unnecessary personal data. A production deployment should define a practical lead-retention period with the business rather than retaining personal data indefinitely.**

# 19. Security Controls

• Database is accessible only from trusted server-side infrastructure.

• Use a dedicated application DB role with least privilege.

• Keep migration/admin credentials separate from runtime credentials.

• Never put DB URLs, passwords or API keys in client-side environment variables.

• Use parameterized queries/safe ORM methods.

• Validate and sanitize all enquiry input server-side.

• Rate-limit the enquiry endpoint.

• Use secure headers and HTTPS in production.

• Avoid unnecessary personal data.

• Do not log raw enquiry messages or secrets.

• Protect backups and staging databases.

• Review dependency/build warnings before production deployment.

These controls directly align with the technical security requirements: HTTPS, server-side validation/sanitization, secret isolation, rate limiting/spam controls, secure headers, minimal personal-data storage and restricted administrative access.

# 20. Migration & Deployment Strategy

Every database change should be delivered as a version-controlled migration.

| PHASE | ACTION |
|---|---|
| Local | Create migration → apply to local DB → seed safe development data → test |
| Preview | Apply same migrations to staging DB → QA forms/content/queries |
| Production | Backup/check → apply migrations → smoke test → monitor |
| Rollback | Use a tested rollback/recovery strategy; avoid destructive migrations without a backup |

## Recommended repository structure

```
db/ ■■■ migrations/ ■■■ seeds/ ■ ■■■ development/ ■ ■■■ staging/ ■■■ schema/ ■■■ README.md src/lib/db/ ■■■ client.ts
■■■ repositories/ ■■■ queries/ ■■■ transactions/
```

The exact folder names can be adapted to the selected ORM or migration tool.

# 21. Database Acceptance Criteria

- All MVP entities are represented by relational tables.
- Primary keys are present and stable.
- Foreign keys protect service/content relationships.
- Public slugs are unique and stable.
- Published/approved states are explicit.
- Gallery records require accessible alt text.
- Reviews cannot appear publicly unless approved.
- Enquiries contain only useful lead fields.
- Enquiry statuses are constrained to known values.
- Indexes support common public reads and lead operations.
- Server-side validation happens before persistence.
- Spam/rate limiting protects the enquiry endpoint.
- Database credentials never reach the browser.
- Development, staging and production databases are separated.
- Migrations are version controlled.
- Seed data contains no fabricated business information.
- Backups and retention are defined before production.
- Production smoke tests cover enquiry insertion and public content reads.

**The project technical acceptance criteria also require working forms over HTTPS, no secret exposure, responsive/accessibility/performance review, valid SEO files and production monitoring/analytics.**

# 22. Final Database Schema Lock

For the MVP, the database should lock to the following core model:

***BUSINESS_PROFILE***
↓
***SERVICES*** ■■→ ***GALLERY_ITEMS***
■
■■■→ ***FAQS***
■
■■■→ ***ENQUIRIES*** ←■■ ***SERVICE_AREAS***

***BUSINESS_PROFILE*** ■■→ ***REVIEWS***
***BUSINESS_PROFILE*** ■■→ ***ANALYTICS_EVENTS***

## Operationally:

***PUBLIC CONTENT → READ-ONLY SERVER QUERIES***
***LEAD FORM → VALIDATE → PROTECT → INSERT → DELIVER***
***ANALYTICS → ALLOWLIST → MINIMAL EVENT DATA***

This schema is intentionally not a full CRM, booking system or CMS. Those can be layered on later without replacing the core business/content/lead model.

DATABASE SCHEMA DOCUMENT v1.0 • Production Baseline